

Flask 飲料商品管理 API 實作題

一、題目名稱

使用 Flask 建立飲料商品管理 API

二、題目說明

請使用 Python 與 Flask 建立一個飲料商品管理 API，將原本只有商品名稱與數量的簡易產品資料，擴充成完整的飲料商品資料。

本題不使用資料庫，所有商品資料暫時儲存在 Python 的 `products` 陣列中。

請依照指定的 API 路徑、HTTP 方法、資料欄位與驗證規則，完成商品的新增、讀取、完整修改、部分修改與刪除功能。

三、商品資料格式

每一筆商品必須包含以下欄位：

```
{
  "pname": "珍珠奶茶",
  "pnum": 8,
  "pstandard": "微冰",
  "psize": "中(500cc-1000cc)",
  "pstatus": "上架",
  "topping": "珍珠",
  "supply": "冷熱皆可",
  "created_at": "2026/7/1 上午9:15:20",
  "id": "P001aBcD001"
}
```

四、欄位規格

欄位名稱	資料型態	說明	是否由前端提供
id	字串	商品編號	否，由後端自動產生
pname	字串	商品名稱	是
pnum	數字	商品數量	是
pstandard	字串	飲料規格，例如微冰、無糖	是
psize	字串	商品容量	是
pstatus	字串	上架狀態	是
topping	字串	加料內容	是
supply	字串	供應方式	是
created_at	字串	商品建立時間	否，由後端自動產生

五、初始資料

請先在 Flask 程式中建立至少 3 筆商品資料：

```
products = [  
  {  
    "pname": "珍珠奶茶",  
    "pnum": 8,  
    "pstandard": "微冰",  
    "psize": "中(500cc-1000cc)",  
    "pstatus": "上架",  
    "topping": "珍珠",  
    "supply": "冷熱皆可",  
    "created_at": "2026/7/1 上午9:15:20",  
    "id": "P001aBcD001"  
  },  
  {  
    "pname": "紅茶",  
    "pnum": 10,  
    "pstandard": "無糖",  
    "psize": "大(1000cc-2000cc)",  
    "pstatus": "上架",  
    "topping": "椰果",  
    "supply": "冷熱皆可",  
    "created_at": "2026/7/1 上午10:22:35",  
    "id": "P002aBcD002"  
  },  
  {  
    "pname": "綠茶",  
    "pnum": 7,  
    "pstandard": "少冰",  
    "psize": "中(500cc-1000cc)",  
    "pstatus": "上架",  
    "topping": "蒟蒻",  
    "supply": "冷飲",  
    "created_at": "2026/7/1 上午11:30:12",  
    "id": "P003aBcD003"  
  }  
]
```

六、API 功能要求

請完成以下 6 個 API。

HTTP 方法	API 路徑	功能
GET	/products	讀取全部商品
GET	/products/<product_id>	讀取指定商品
POST	/products	新增商品
PUT	/products/<product_id>	完整修改指定商品
PATCH	/products/<product_id>	部分修改指定商品
DELETE	/products/<product_id>	刪除指定商品

因為商品編號為字串，所以路由參數應使用字串格式，例如：

```
@app.route("/products/<string:product_id>", methods=["GET"])
```

七、各 API 詳細要求

1. 讀取全部商品

API

GET /products

功能要求

回傳 `products` 陣列中的全部商品資料。

成功回傳範例

```
{
  "message": "資料讀取成功",
  "data": [
    {
      "pname": "珍珠奶茶",
      "pnum": 8,
      "pstandard": "微冰",
      "psize": "中(500cc-1000cc)",
      "pstatus": "上架",
      "topping": "珍珠",
      "supply": "冷熱皆可",
      "created_at": "2026/7/1 上午9:15:20",
      "id": "P001aBcD001"
    }
  ]
}
```

2. 讀取指定商品

API

GET /products/<product_id>

測試網址

GET http://127.0.0.1:5000/products/P001aBcD001

功能要求

依照網址中的 `product_id` 尋找指定商品。

找到商品時，回傳該筆商品資料。

成功回傳範例

```
{
  "message": "資料讀取成功",
  "product": {
    "pname": "珍珠奶茶",
    "pnum": 8,
    "pstandard": "微冰",
    "psize": "中(500cc-1000cc)",
    "pstatus": "上架",
    "topping": "珍珠",
    "supply": "冷熱皆可",
    "created_at": "2026/7/1 上午9:15:20",
    "id": "P001aBcD001"
  }
}
```

找不到商品時，回傳錯誤訊息。

```
{
  "message": "找不到此產品"
}
```

3. 新增商品

API

POST /products

請求資料範例

```
{
  "pname": "黑糖奶茶",
  "pnum": 9,
  "pstandard": "半糖",
  "psize": "小(500cc以下)",
  "pstatus": "上架",
  "topping": "珍珠/布丁",
  "supply": "熱飲"
}
```

功能要求

新增商品時，前端只需要傳送以下欄位：

```
pname
pnum
pstandard
psize
pstatus
topping
supply
```

以下欄位由後端自動產生：

```
id
created_at
```

商品編號產生規則

商品編號格式如下：

```
P001aBcD001
P002aBcD002
P003aBcD003
```

新增商品時，請取得目前最大的商品流水號，再加 1。

例如目前最大商品編號為：

P030aBcD030

下一筆商品編號應為：

P031aBcD031

不可單純使用 `len(products) + 1`，避免刪除商品後產生重複編號。

建立時間

`created_at` 必須由後端取得目前時間後自動產生。

成功回傳範例

```
{
  "message": "產品新增成功",
  "product": {
    "pname": "黑糖奶茶",
    "pnum": 9,
    "pstandard": "半糖",
    "psize": "小(500cc以下)",
    "pstatus": "上架",
    "topping": "珍珠/布丁",
    "supply": "熱飲",
    "created_at": "系統目前時間",
    "id": "P004aBcD004"
  },
  "products": []
}
```

4. 完整修改商品

API

PUT /products/<product_id>

測試網址

PUT http://127.0.0.1:5000/products/P001aBcD001

請求資料範例

```
{
  "pname": "特濃珍珠奶茶",
  "pnum": 15,
  "pstandard": "少冰",
  "psize": "大(1000cc-2000cc)",
  "pstatus": "上架",
  "topping": "珍珠/布丁",
  "supply": "冷熱皆可"
}
```

功能要求

PUT 為完整更新，因此必須提供全部可修改欄位：

```
pname
pnum
pstandard
psize
pstatus
topping
supply
```

PUT 不可修改：

```
id
created_at
```

更新成功後，原本的 `id` 與 `created_at` 必須保留。

成功回傳範例

```
{  
  "message": "產品更新成功。",  
  "product": {},  
  "products": []  
}
```

5. 部分修改商品

API

PATCH /products/<product_id>

測試網址

PATCH http://127.0.0.1:5000/products/P001aBcD001

請求資料範例

只修改商品數量：

```
{  
  "pnum": 20  
}
```

同時修改商品數量與狀態：

```
{  
  "pnum": 20,  
  "pstatus": "下架"  
}
```

功能要求

PATCH 只修改前端有傳送的欄位。

未傳送的欄位必須保留原本資料。

PATCH 不可修改：

```
id
created_at
```

若請求資料完全沒有可修改的商品欄位，應回傳錯誤訊息。

成功回傳範例

```
{
  "message": "產品更新成功。",
  "product": {},
  "products": []
}
```

6. 刪除商品

API

```
DELETE /products/<product_id>
```

測試網址

```
DELETE http://127.0.0.1:5000/products/P001aBcD001
```

功能要求

依照 `product_id` 尋找指定商品，找到後將商品從 `products` 陣列中刪除。

成功回傳範例

```
{  
  "message": "產品刪除成功。",  
  "product": {},  
  "products": []  
}
```

找不到商品時，回傳：

```
{  
  "message": "找不到此產品"  
}
```

八、資料驗證規則

1. JSON 格式驗證

當使用 POST、PUT 或 PATCH 時，必須先確認前端是否傳送正確的 JSON 格式。

錯誤回傳：

```
{  
  "error": "請傳送正確的json格式資料"  
}
```

HTTP 狀態碼：

400

2. 商品名稱 pname

- 必須提供商品名稱。

- 必須是文字格式。
- 不得為空字串。
- 商品名稱不得與其他商品重複。
- PUT 或 PATCH 修改自己原本的名稱，不算重複。

錯誤回傳範例：

```
{  
  "error": "產品名稱必須是文字格式"  
}
```

```
{  
  "message": "產品名稱存在，請重新命名。"  
}
```

3. 商品數量 `pnum`

- 必須是數字格式。
- 不得小於 0。
- 數量為 0 時應視為有效資料，不可使用 `if not pnum` 判斷。

錯誤回傳範例：

```
{  
  "error": "產品數量必須是數字格式"  
}
```

```
{  
  "error": "產品數量必須不能是負數"  
}
```

4. 商品容量 `psize`

`psize` 只能使用以下其中一個值：

小(500cc以下)
中(500cc-1000cc)
大(1000cc-2000cc)
特大(2000cc以上)

5. 商品狀態 `pstatus`

`pstatus` 只能使用以下其中一個值：

上架
下架

6. 供應方式 `supply`

`supply` 只能使用以下其中一個值：

冷飲
熱飲
冷熱皆可

7. 其他文字欄位

以下欄位必須是文字格式，且不得空白：

`pstandard`
`topping`

九、HTTP 狀態碼要求

執行結果	狀態碼
讀取成功	200
新增成功	201
修改成功	200
刪除成功	200
JSON 格式錯誤	400
欄位驗證失敗	400
找不到指定商品	404

十、程式撰寫要求

1. 使用 Flask 建立 API。
2. 使用 `request.get_json()` 取得前端資料。
3. 使用 `jsonify()` 回傳 JSON。
4. 商品資料儲存在 `products` 陣列中。
5. 不使用 MySQL、SQLite 或其他資料庫。
6. 每一個 API 必須指定正確的 HTTP 方法。
7. 成功與失敗皆必須回傳適當的 HTTP 狀態碼。
8. 不可直接信任前端資料，必須完成指定欄位驗證。
9. PUT 必須完整更新資料。
10. PATCH 必須只更新有傳送的欄位。
11. PUT、PATCH 不得修改 `id` 與 `created_at`。
12. 程式必須可以透過 Postman 完成測試。

十一、Postman 測試項目

完成後，請至少測試以下情況：

GET 測試

- 讀取全部商品。
- 讀取存在的商品。
- 讀取不存在的商品。

POST 測試

- 正常新增商品。
- 傳送錯誤 JSON。
- 缺少必要欄位。
- 商品名稱重複。
- 商品數量為負數。
- 商品數量為 0。
- 商品狀態填入錯誤內容。
- 供應方式填入錯誤內容。

PUT 測試

- 正常完整修改商品。
- 缺少其中一個必要欄位。
- 修改不存在的商品。
- 修改成其他商品已使用的名稱。

PATCH 測試

- 只修改一個欄位。
- 同時修改多個欄位。
- 傳送空 JSON。
- 修改不存在的商品。
- 嘗試修改 `id` 或 `created_at`。

DELETE 測試

- 刪除存在的商品。
- 刪除不存在的商品。
- 刪除後重新讀取全部商品，確認資料已移除。

十二、繳交內容

請繳交以下檔案：

`app.py`

並附上 Postman 測試成功的畫面，至少包含：

1. GET 全部商品。
2. GET 單一商品。
3. POST 新增商品。
4. PUT 完整修改商品。
5. PATCH 部分修改商品。
6. DELETE 刪除商品。
7. 至少 3 種資料驗證失敗的測試結果。

十三、完成標準

完成後，程式應具備以下功能：

- 可以讀取全部商品。
- 可以透過商品編號讀取單一商品。
- 可以新增完整商品資料。
- 可以自動產生商品編號。
- 可以自動產生建立時間。
- 可以完整修改商品。
- 可以部分修改商品。

- 可以刪除商品。
- 可以阻擋錯誤資料。
- 可以阻擋重複商品名稱。
- 可以回傳正確的 JSON 與 HTTP 狀態碼。